Continuously Evolving AI Agent Architecture

Implementation with Claude Code, .claude/agents, and a controlled evolution system

Prepared from the conversation: how to implement a continuously evolving agent and which architecture is better.

1. Can continuous evolution be implemented in .claude/agents?

Yes. Claude Code agent definitions can be used as part of a continuous-improvement workflow. The important distinction is between improving agent instructions and skills, changing actual software code, and changing the underlying model weights. Agent files can support the first two through a controlled development process; they do not independently train or modify the underlying model.

Suggested project structure

```
.claude/
■■■ agents/
■   ■■■ orchestrator.md
■   ■■■ researcher.md
■   ■■■ architect.md
■   ■■■ developer.md
■   ■■■ tester.md
■   ■■■ reviewer.md
■   ■■■ evolution-agent.md
■■■ skills/
■   ■■■ code-optimization/
■   ■■■ self-evaluation/
■   ■■■ continuous-improvement/
■■■ rules/
    ■■■ evolution-policy.md

.agent-evolution/
■■■ memory/
■   ■■■ lessons.md
■   ■■■ successful-patterns.md
■■■ evaluations/
■   ■■■ benchmark.json
■   ■■■ history.json
■■■ proposals/
■■■ versions/
```

The .claude/agents directory contains Claude Code agent definitions. The .agent-evolution directory is an example custom project structure for storing evaluation results, proposals, and versions; it is not a built-in Claude Code feature.

2. Continuous evolution loop

```
Receive task
   ↓
Orchestrator plans and delegates
   ↓
Research / architecture / implementation
   ↓
Testing and code review
   ↓
Evaluation
   ■■■ Pass → record successful patterns
   ■■■ Fail → analyze failure
                  ↓
            Evolution agent
                  ↓
            Propose improvement
                  ↓
         Benchmark + regression tests
                  ↓
           Accept or reject
                  ↓
         Version / rollback support
                  ↓
```

```
          Future development
```

The evolution agent should not assume that every change is an improvement. A candidate must be evaluated against existing tasks and tests before it is promoted.

3. Example Evolution Agent definition

```
---
name: evolution-agent
description: Analyzes development results and proposes validated improvements.
---

# Mission
Improve quality, efficiency, reliability, and capability
through evidence-based evolution.

# Responsibilities
1. Analyze completed tasks.
2. Examine test failures and review feedback.
3. Identify recurring problems and inefficient workflows.
4. Propose improvements to instructions, skills, workflows, and code.
5. Define validation tests.
6. Record successful improvements and failed experiments.

# Process
1. Collect task specification and results.
2. Inspect implementation and tests.
3. Identify bottlenecks and root causes.
4. Generate improvement proposals.
5. Estimate benefits and risks.
6. Validate candidates against benchmarks.
7. Record results and recommend promotion.

# Safety
- Never overwrite the original configuration directly.
- Preserve previous working versions.
- Require tests before accepting code changes.
- Keep an audit trail.
- Make changes reversible.
- Do not modify credentials or security policies.

# Output
Problem, evidence, root cause, proposed change, expected benefit,
risk, validation plan, evaluation result, and version recommendation.
```

4. Example Orchestrator definition

```
---
name: orchestrator
description: Coordinates development and continuous improvement.
---

# Agent team
- researcher: investigates technical information
- architect: designs system architecture
- developer: implements software
- tester: validates functionality
- reviewer: reviews quality
- evolution-agent: analyzes results and proposes improvements

# Workflow
1. Understand the objective.
2. Break it into tasks.
3. Assign specialized agents.
4. Collect implementation results.
5. Run tests and reviews.
6. Evaluate the completed work.
7. Ask evolution-agent to analyze results.
8. Record approved lessons.
9. Update the development plan.

# Continuous improvement
Identify recurring failures and successful patterns, review agent
performance, propose workflow improvements, evaluate candidates,
```

```
and preserve previous configurations.
```

A configuration file is not itself a persistent process. An agent reads instructions when invoked; editing a file does not guarantee that an already-running agent reloads its instructions immediately. Prefer candidate files and future invocations.

5. Three levels of evolution

Level	What changes	Example
1. Memory evolution	Lessons, successful patterns, known mistakes	Record a testing strategy that catches recurring bugs.
2. Configuration evolution	Instructions, skills, task decomposition, workflows	Require benchmarks before accepting optimization work.
3. Code evolution	Implementation, algorithms, architecture	Replace a slow search algorithm and benchmark it.

For a robust organization, combine all three levels.

6. How to implement self-modification safely

```
Evolution Agent
  ↓
Read current configuration / code
  ↓
Analyze task history and evaluation results
  ↓
Generate candidate improvement
  ↓
Write candidate version (not production)
  ↓
Run tests, benchmarks, regression and security checks
  ↓
Compare with current version
  ↓
Accept → versioned promotion
Reject → retain current version
```

Suggested candidate layout:

```
.agent-evolution/
■■■ candidates/
■   ■■■ developer-v2.md
■■■ evaluations/
■   ■■■ developer-v2-results.json
■■■ versions/
    ■■■ developer-v1.md
    ■■■ developer-v2.md
```

Evaluation metrics

• Test pass rate and task completion rate

• Code quality and maintainability

• Execution time and token consumption

• Regression rate

• Security checks

• Reproducibility of results

The candidate should show measurable improvement, not merely produce different or shorter code.

7. Architecture comparison

Option A — Claude Code above Evolution Agent

```
Claude Code
```

```
  ■■■ Research Agent
  ■■■ Developer Agent
  ■■■ Tester Agent
  ■■■ Evolution Agent
        ■■■ improves instructions / workflows
```

Pros: simpler to implement; evolution can be added as a specialized agent; good for a prototype.

Cons: evolution is subordinate to the development system and may focus only on its current workflow rather than the entire organization.

Option B — Evolution System above Claude Code

```
Continuously Evolving System
  ■■■ Evolution Controller
  ■■■ Performance Evaluator
  ■■■ Knowledge / Memory
  ■■■ Experiment Manager
  ■■■ Claude Code Agent
        ■■■ Research
        ■■■ Architecture
        ■■■ Development
        ■■■ Testing
        ■■■ Review
```

Pros: evolution manages the entire organization; evaluates workflows and agents; can improve configuration, skills, tools, software, and task allocation; supports versioning and rollback.

Cons: requires more architecture, evaluation infrastructure, and permission controls.

Recommendation: Option B is better for the intended long-term autonomous software development organization. Option A is easier for a small prototype.

8. Recommended production architecture

```
Continuously Evolving System
  |
  ■■■ Evolution Controller
  ■■■ Performance Evaluator
  ■■■ Knowledge / Memory
  ■■■ Experiment Manager
  |
  ■■■ Claude Code Runtime
        ■■■ Research Agent
        ■■■ Architect Agent
        ■■■ Developer Agent
        ■■■ Tester Agent
        ■■■ Reviewer Agent
              |
              v
        Development Results
              |
              v
        Evaluation Engine
              |
              v
        Evolution Proposals
              |
              v
        Benchmarks + Regression Tests
              |
          Approved?
          /      \
        Yes      No
         |        |
   Versioned     Retain
   Agent Update  Current Version
         \        /
          Future Work
```

Core principle: Claude Code performs development. The Evolution Controller manages how Claude Code develops.

9. Responsibility boundaries

Component	Responsibility
Evolution Controller	Evolution strategy, experiment management, approval, promotion, rollback
Claude Code	Executes software development tasks
Evolution Agent	Researches, analyzes, and proposes improvements
Evaluator	Determines whether proposed changes actually work
Memory	Stores knowledge and historical evaluation results

10. Final implementation recommendation

• Build a separate Evolution Controller.

• Use Claude Code as the software development runtime.

• Use .claude/agents/ for specialized Claude Code agents.

• Use an Evolution Agent to analyze and propose improvements.

• Use an evaluator to validate candidates.

• Promote tested versions and preserve rollback capability.

Conclusion: The strongest design is not to make Claude Code the evolution system. Make Claude Code a development engine managed by a continuously evolving system.